

HW/SW Co-Design Project

Benchmark report: raytrace

pyperformance optimization and hardware acceleration (V3A)

Software speedup	Run time	Correctness	Hardware (V3A)
2.85x	797 ms -> 279 ms	bit-identical image	1 test/cycle, ~1.7x

Environment: Linux-5.15.0-1106-kvm-x86_64-with-glibc2.35, 1 CPU(s), Python 3.10.12 (64-bit), pypyperformance 1.14.0.

1. Overview

Purpose

raytrace is a small, pure-Python recursive (Whitted-style) ray tracer by Callum and Tony Garnock-Jones (2008). It renders a fixed 100x100 scene: seven spheres over a checkerboard floor, lit by two point lights, with specular reflections, Lambert diffuse shading with shadows, and ambient light. It is a classic "object-oriented numeric Python" workload: millions of tiny objects and short method calls, which stresses CPython's function call, attribute access and allocation paths far more than the arithmetic.

Libraries

- math (sqrt, tan, pi) and array (array('B') holds the RGB canvas).
- pyperf: timing harness only; not part of the measured computation.

Data structures

- Vector, Point : classes with three float attributes x, y, z stored in a per-instance `__dict__`. Vector/Point arithmetic (`__add__`, `__sub__`, `scale`, `dot`, `cross`, `normalized`) always returns NEW objects.
- Ray : origin Point + normalized direction Vector.
- Sphere(centre, radius), Halfspace(point, normal) : scene objects with `intersectionTime(ray)` and `normalAt(p)`.
- SimpleSurface / CheckerboardSurface : material (base colour, specular, Lambert and ambient coefficients).
- Scene : list of (object, surface) tuples, list of light Points, camera; Canvas : flat array('B') of width*height*3 bytes.

Algorithm (per frame)

```
for each of the 10,000 pixels:
    build a primary ray from the camera through the pixel (normalize)
    rayColour(ray):
        intersect the ray with ALL 8 objects, keep the closest hit t > -eps
        p = hit point, n = surface normal at p
        specular : reflect the ray about n and recurse
        diffuse  : for each light, cast a shadow ray from p towards the light
                  and test it against the objects; add max(0, dir . n)
        ambient  : constant term
    clamp colour and write 3 bytes into the canvas
Ray-sphere test: with cp = centre - origin and v = cp . d,
discriminant = r^2 - (|cp|^2 - v^2); hit time = v - sqrt(discriminant).
```

Why this is slow in CPython

Every vector operation is a Python method call (frame creation, argument binding) that allocates a new object with its own `__dict__`. The original code also calls "type assertion" methods (`mustBeVector()`, `isPoint()`) that do nothing for valid input but still cost a full call each, and it rebuilds an identical shadow Ray (with a sqrt) for every object it tests.

2. Initial analysis (profiling)

Method (all automated in script_raytrace.sh)

- pyperformance run -b raytrace -> results/baseline.json
- perf stat -r 5 (instructions, branches, cache) -> results/perf_stat_original.txt
- perf record -e cpu-clock -F 999 -g -- python3-dbg -m pyperformance run
- --bench raytrace (course guide method) -> results/perf_report_original*.txt
- FlameGraph (stackcollapse-perf.pl | flamegraph.pl) -> flamegraphs/flamegraph_raytrace_original.svg
- callcount_raytrace.py (Python calls and object allocations per render)

python3-dbg lets perf resolve CPython's internal C symbols; note that Python 3.10 has no `-X perf`` support, so the flame graph shows interpreter C functions, not Python-level functions. VM note: the QEMU vCPU exposes no "cycles" event, so we sample on cpu-clock. perf stat runs profile_raytrace.py (5 renders in one process) so counters exclude pyperf worker spawning and venv setup.

Baseline result

pyperformance run -b raytrace: 797 ms +- 5.8 ms per render (60 values)
(under python3-dbg while profiling: 2175 ms +- 24.3 ms)

perf report -- hottest symbols, self overhead (original)

```
overhead symbol
26.58% _PyEval_EvalFrameDefault
3.30% call_function.lto_priv.0
2.98% frame_dealloc.lto_priv.0
2.72% _PyEval_MakeFrameVector
1.98% lookdict_unicode_nodummy
1.97% _PyDict_GetItemHint
1.96% PyTuple_GetItem
1.93% binary_op1
1.88% _PyType_Lookup
```

```

1.77% _PyObject_VectorcallTstate.1to_priv.10
1.57% _PyObject_GetMethod
1.57% lookdict_split.1to_priv.0
1.53% _PyMem_DebugCheckAddress
1.14% _PyEval_Vector
1.12% pthread_getspecific@@GLIBC_2.34

```

grouped by role:

```

26.58% bytecode dispatch (_PyEval_EvalFrameDefault)
16.31% Python function calls / frames
14.67% attribute access (getattr/setattr, dict, slots)
5.60% object creation / destruction
4.02% float objects: arithmetic + alloc/free
0.00% math: sqrt / libm
3.40% number-protocol dispatch
3.48% tuple/list handling
6.27% debug-build self-checks (python3-dbg only)
(99.3% of all samples are in the benchmark worker process; the rest
is pyperformance start-up.)

```

perf stat (original), per run of 5 renders

```

instructions      25,641,141,365
branches          4,945,496,813
branch-misses     14,637,980    0.30% of branches
cache-references  13,323,843
cache-misses      34,366        0.26% of cache refs
task-clock        4061.4 ms
cycles            not exposed by the VM

```

Interpretation: cache misses are negligible (0.26% of cache references) and branch misses are negligible (0.30%). One render executes ~5.13 billion instructions for 2,882,307 Python function calls, i.e. ~1,779 machine instructions per Python-level call. The cost is dominated by call and object overhead, not by memory access or the floating-point math itself.

Python function calls per render (callcount_raytrace.py, original)

```

Vector.mustBeVector      520,539    18.1%
Vector.dot                509,871    17.7%
Vector.__init__           452,943    15.7%
Point.__sub__             277,865     9.6%
Point.isPoint            277,865     9.6%
Sphere.intersectionTime   179,457     6.2%
Vector.scale              149,743     5.2%
Vector.normalized         109,887     3.8%
Vector.magnitude          109,887     3.8%
Ray.__init__              98,172     3.4%
Halfspace.intersectionTime 25,501     0.9%
Vector.__add__            20,000     0.7%
Vector.isPoint            20,000     0.7%
addColours                15,999     0.6%
Scene.rayColour           15,333     0.5%
<listcomp>                15,265     0.5%
TOTAL                    2,882,307

```

objects allocated per render (original):

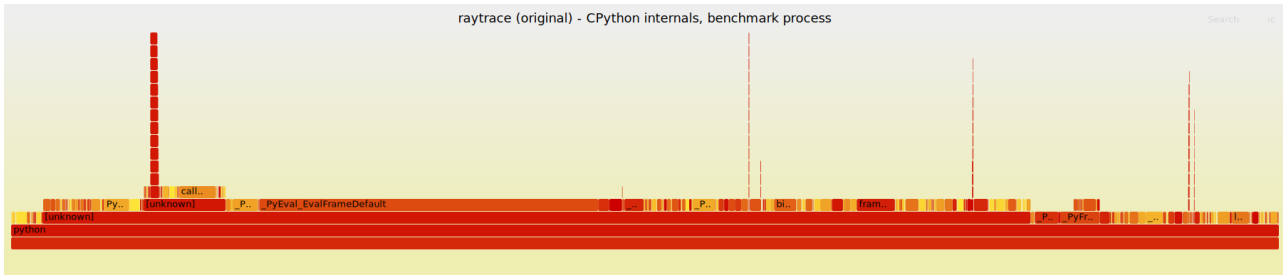
```

Vector      452,943
Point        5,345
Ray         98,172

```

Bottlenecks identified

- B1 Type-check calls: 818,411 calls per render (28% of all Python calls) to mustBeVector()/isPoint(), which do nothing for valid input.
- B2 Temporary objects: 452,943 Vector objects per render, most of them intermediates of expressions like (centre - origin).dot(direction) that live for a single operation; each needs an allocation, a `__dict__` and a deallocation.
- B3 Redundant work in the shadow test: `_lightIsVisible()` is called 10,666 times but builds a new, identical Ray (sqrt + normalize) for every object it tests, so the render creates 98,172 Rays although only ~25,999 distinct rays are needed.
- B4 Attribute access through per-instance `__dict__` on every `.x/.y/.z` read and write.



Flame graph - raytrace, original (perf record -e cpu-clock -g, python3-dbg). Interactive version: [benchmarks/raytrace/flamegraphs/flamegraph_raytrace_original.svg](#)

3. Optimizations

All changes are in benchmarks/raytrace/optimized/run_benchmark.py. The scene, image size, algorithm, public classes/methods and the benchmark name are unchanged, so it runs under pyperformance through a custom manifest (benchmarks/raytrace/manifest/MANIFEST). Every floating-point expression keeps the original operand order, so the rendered image is BIT-IDENTICAL.

- 01 `__slots__` on Vector, Point and Ray (targets B4, B2)
Attributes live at fixed offsets in the object (slot descriptors) instead of a per-instance dict: faster `.x/.y/.z` access and cheaper creation.
- 02 Remove type-check calls from the hot path (targets B1)
`mustBeVector()/isPoint()` calls are dropped; where the result type really depends on the argument (`Vector + Point`, `Point - Point`) a direct class comparison (`other.__class__ is Point`) replaces the method call.
- 03 Loop-invariant code motion in the shadow test (targets B3)
`_lightIsVisible()` builds `Ray(p, light - p)` once per light instead of once per tested object. The ray does not depend on the object, so the result is identical.
- 04 Inline hot vector math on floats (targets B2)
`Sphere/Halfspace.intersectionTime`, `normalAt`, Ray construction (normalization), `pointAtTime`, `reflectThrough`, the primary-ray direction in `render()` and the Lambert term in `colourAt()` compute directly on local floats instead of chaining Vector method calls that create temporaries. The closest-hit search no longer builds a list of tuples.

```
Correctness (verify raytrace.py)
image 100x100 (30000 bytes)
sha256 original : 11f2c36b02044f115c3937cae1156e9528b95da70f9d6b737a262f06757c2e6b
sha256 optimized: 11f2c36b02044f115c3937cae1156e9528b95da70f9d6b737a262f06757c2e6b
differing bytes : 0
ablation stage v0 original      : identical
ablation stage v1_no_checks     : identical
ablation stage v2_hoist         : identical
ablation stage v3_inline        : identical
ablation stage v4_slots         : identical
PASS (bit-identical)
```

function	original	optimized	
Vector.mustBeVector	520,539	0	
Vector.__init__	452,943	57,338	
Vector.dot	509,871	0	
Sphere.intersectionTime	179,457	179,457	
Point.__sub__	277,865	14,524	
Point.isPoint	277,865	0	
Vector.scale	149,743	0	
Ray.__init__	98,172	26,000	
Vector.normalized	109,887	3	
Vector.magnitude	109,887	0	
Halfspace.intersectionTime	25,501	25,501	
addColours	15,999	15,999	
Scene.rayColour	15,333	15,333	
Scene._lightIsVisible	10,666	10,666	
Vector.__add__	20,000	0	
Vector.isPoint	20,000	0	
Canvas.plot	10,000	10,000	
<listcomp>	15,265	0	
TOTAL calls	2,882,307	392,204	(-86%)
Vector objects	452,943	57,338	(-87%)
Point objects	5,345	5,345	(+0%)
Ray objects	98,172	26,000	(-74%)

The amount of real ray-tracing work is unchanged (`Sphere.intersectionTime` is called 179,457 vs 179,457 times); what disappears is the call and allocation overhead around it.

Considered and not pursued

- numpy vectorization: each operation works on a single 3-vector per ray and the ray tree is recursive and data-dependent, so numpy's per-call overhead would dominate (as measured for nbody), and vectorizing would change the floating-point evaluation order (no bit-identical image).
- Changing the scene, image size or recursion depth: would change the workload, not optimize it.

4. Performance comparison

pyperformance (results/compare.txt)

original : 797 ms +- 5.8 ms
 optimized: 279 ms +- 2.8 ms
 -> 2.85x faster, run time -65% (meets the >= 7% requirement)
 Same trend under python3-dbg while profiling: 2175 ms -> 626 ms (3.48x).

Ablation (ablation_raytrace.py) -- optimizations added one at a time

stage (cumulative)	time / render	vs original	this step
original	806.4 +- 8 ms	1.00x	1.00x
+ O2 remove type-check calls	737.1 +- 4 ms	1.09x	1.09x
+ O3 hoist shadow ray	566.1 +- 4 ms	1.42x	1.30x
+ O4 inline vector math	307.9 +- 2 ms	2.62x	1.84x
+ O1 __slots__ (final)	277.7 +- 2 ms	2.90x	1.11x

Every stage renders a bit-identical image (checked in verify_raytrace.py).

perf stat before/after (5 renders per run, mean of 5 runs)

	original	optimized	change
instructions	25,641,141,365	9,364,171,653	-63.5%
branches	4,945,496,813	1,764,501,736	-64.3%
branch-misses	14,637,980	8,804,429	-39.9%
cache-misses	34,366	31,698	-7.8%
page-faults	2,836	2,921	+3.0%
task-clock	4061.4 ms	1472.6 ms	2.76x faster
elapsed	4.104 s	1.478 s	2.78x faster

Instructions drop by 63% while Python calls drop by 86% and Vector allocations by 87%. The instruction ratio (2.74x) and the time ratio (2.76x) agree: the CPU simply executes far less interpreter work for the same image.

perf profile before/after (flame graphs: flamegraphs/flamegraph_raytrace_*.svg)

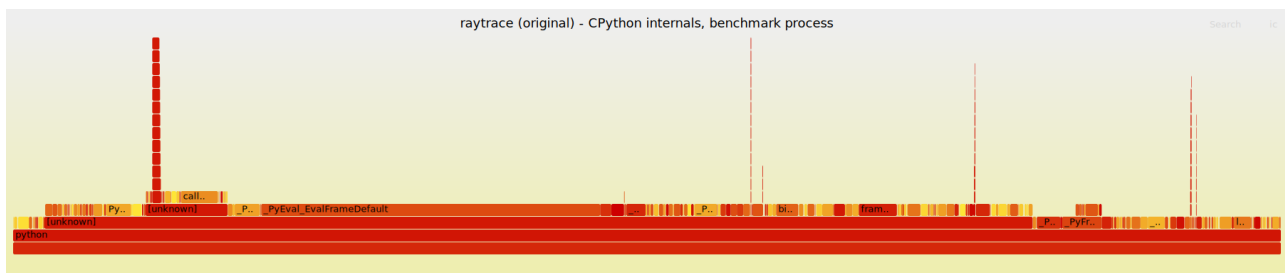
Estimated absolute cost = self% x total sampled time of each profile.

role	original	optimized	change
bytecode dispatch (PyEval EvalFrameDefault)	26.6%	28.8%	-60%
Python function calls / frames	16.3%	7.9%	-82%
attribute access (getattr/setattr, dict, slots)	14.7%	6.7%	-83%
object creation / destruction	5.6%	6.6%	-57%
float objects: arithmetic + alloc/free	4.0%	7.5%	-31%
math: sqrt / libm	0.0%	0.0%	n/a
number-protocol dispatch	3.4%	4.9%	-46%
tuple/list handling	3.5%	3.1%	-67%
debug-build self-checks (python3-dbg only)	6.3%	7.3%	-57%

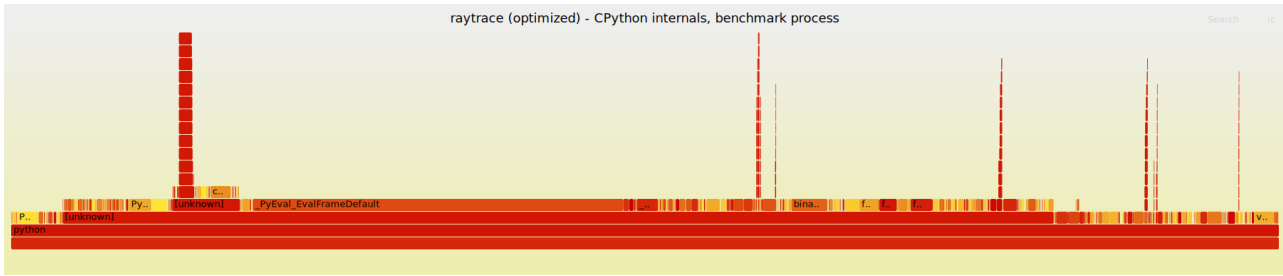
Largest absolute reductions: PyEval EvalFrameDefault (-60%), call_function.lto_priv.0 (-78%), frame_dealloc.lto_priv.0 (-79%), PyEval MakeFrameVector (-81%), PyDict GetItemHint (-90%), PyType Lookup (-80%).

Top of the optimized profile: PyEval EvalFrameDefault (28.8%), binary_op1 (3.0%), PyFloat FromDouble (1.9%), call_function.lto_priv.0 (1.9%), PyTuple GetItem (1.7%).

Total sampled time: original 81.20 s, optimized 30.05 s (cpu-clock event count, includes pyperformance start-up).



Flame graph - raytrace, BEFORE optimization



Flame graph - raytrace, AFTER optimization. Interactive version:
[benchmarks/raytrace/flamegraphs/flamegraph_raytrace_optimized.svg](#)

5. Hardware acceleration proposal

One accelerator serves both benchmarks: V3A, a 3D Vector-math Accelerator. The hot arithmetic of nbody (pair distance, $|d|^{\wedge}3$) and of raytrace (ray-sphere test: dot products and a square root) is the same small set of operations: 3D subtract, dot product and inverse square root. V3A implements them once as reusable pipelined IP and exposes two kernels on one PCIe card. Full RTL: `hw/rtl/*.v` (Verilog-2005/SystemVerilog-compatible, Icarus-tested).

5.1 Hardware description (hw/rtl)

```
v3a_top.v           PCIe core logic: register bus decode, IRQ
v3a_nbody_ctrl.v    nbody kernel: body registers, step FSM, integrator
v3a_pair_pipe.v      pairwise gravity datapath, 19 stages
v3a_ray_ctrl.v       ray kernel: ray BRAM, feed FSM, reducer, results
v3a_raysphere_pipe.v ray-sphere datapath, 19 stages
v3a_invsgqrt.v       shared inverse-square-root IP, 12 stages
v3a_delay.v          sideband pipeline balancing
v3a_common.vh        Q23.40 fixed-point multiply
v3a_invsgqrt_lut.vh  64-entry initial-guess ROM (generated)
Golden model: hw/model/v3a_model.py (bit-accurate). Testbench:
hw/tb/tb_v3a.v. Everything is run by: python3 hw/run_hw_tests.py
```

5.2 Inputs and outputs

```
Number format : signed Q23.40, 64 bits (range +-8.4e6, LSB 9.1e-13).
                  Chosen instead of IEEE-754 double: a fixed-point multiply
                  needs no normalization/rounding logic (about half the area
                  of a double-precision FPU multiplier, one stage per op).
Clock           : single clock, target 100 MHz (see 5.7), sync. reset.
Host bus        : memory-mapped, word-addressed, 64-bit data (PCIe BAR0
                  behind an AXI4-Lite bridge; BAR byte offset = 8 x index).
                  24-bit address: 0x000000 nbody window, 0x100000 ray window.
Interrupt       : irq (level) when a job is done -> MSI.
nbody kernel    : in : NBODIES (2..8), NSTEPS, DT, per body x,y,z,vx,vy,vz,m*dt
                  out: updated x,y,z,vx,vy,vz after NSTEPS steps; STEPDONE,
                  CYCLES (performance counter)
ray kernel      : in : NRAYS (1..4096), NSPH (1..16), EPS, MODE
                  (0 closest hit, 1 any hit), spheres c,r^2,
                  rays p,d (d normalized)
                  out: per ray (sphere index or -1, t) / (occluded 1/0)
Pipeline IP     : valid/data streaming, no back-pressure needed (the
                  controllers always accept every output).
v3a_invsgqrt    in  x (Q23.40)                out y = x^-1/2
v3a_pair_pipe   in  r_i, r_j, m_i*dt, m_j*dt, i, j out f_i, f_j, i, j
v3a_raysphere_pipe in  p, d, c, r^2, ray id, sph id out disc, t, ids
```

5.3 Hardware architecture

Datapath (see `hw/docs/v3a_pipelines.png`):

- Every stage has at most ONE 64x64 multiply followed by a register, so the critical path is the same everywhere and throughput is one result per clock cycle after the 19-cycle latency.
- v3a invsgqrt: stage 1 normalizes $x = m \cdot 4^k$ with m in $[1,4)$ using a priority encoder and a barrel shift; stage 2 takes y_0 from a 64-entry ROM indexed by the top 6 bits of m ; stages 3-11 run three Newton iterations $y \leftarrow y(3 - m y^2)/2$ (3 stages each: y^2 , $m y^2$, update); stage 12 shifts back by k . Measured accuracy: relative error $< 2.3e-7$ over $1e-6..1e6$ (limited by the Q23.40 LSB), $\sim 1e-12$ for typical values.
- Sideband data (indices, d , masses, v) travel through v3a_delay shift registers so they stay aligned with the invsgqrt result.

Control logic:

- v3a nbody_ctrl FSM per time step: CLR (zero accumulators) -> FEED (one pair $(i < j)$ per cycle into the pipeline) -> WAIT (outputs accumulate $acc_i \leftarrow f_i$, $acc_j \leftarrow f_j$) -> VEL ($v \leftarrow acc$ for all bodies) -> POS ($r \leftarrow dt \cdot v$, one body per cycle) -> NEXT (repeat NSTEPS times -> IRQ).

All pairs of a step use the positions from the start of the step, so

```

pairs are independent and can be streamed through the pipeline.
- v3a ray ctrl: a feed FSM walks (ray r, sphere s) and streams one test
per cycle (ray words come from a synchronous BRAM, 1-cycle read); a
reducer after the pipeline keeps the running best hit of the current
ray (t > -EPS and smallest t; or any t > EPS for shadow rays) and writes
the result BRAM when the last sphere of that ray comes out.
Verification (hw/run_hw_tests.py, vectors from the real benchmark code):
nbody : 5 bodies, 2000 steps, 10 pairs/step
RTL vs golden fixed-point model: BIT-EXACT
energy: float benchmark -0.169071606870, hardware -0.169071606646
relative difference 1.33e-09
max |position diff| vs float benchmark: 9.01e-07
cycles: 76000 = 38.0 per step (10 pair feeds +
19 latency + 5 position updates + control)
ray : closest hit, primary rays
2500 rays x 7 spheres = 17500 tests
RTL vs golden fixed-point model: BIT-EXACT
closest sphere == float benchmark: 2500/2500 rays
(194 hits), max |t error| 1.12e-09
cycles: 17521 = 1.001 per test (+19 latency, +1 BRAM read)
ray : any hit, shadow rays
1940 rays x 7 spheres = 13580 tests
RTL vs golden fixed-point model: BIT-EXACT
shadow decision == float benchmark: 1940/1940 rays
(53 occluded)
cycles: 13601 = 1.002 per test (+19 latency, +1 BRAM read)
RESULT: PASS

```

5.4 Hardware/software interface

```

Driver      : Linux PCIe driver, character device /dev/v3a. mmap() of BAR0
              for the registers; pinned DMA buffers for bulk data; MSI
              interrupt wakes the waiting process.
Transfers   : small state (nbody: 5 bodies x 7 words = 280 bytes) is written
              directly through BAR0. Ray batches (48 bytes per ray, up to
              4096 rays) are copied by the card's AXI4 DMA master from a
              pinned host buffer into the ray BRAM; results are copied back
              the same way (the RTL models the BRAM filled via the bus).
API         : Python extension module (C, or ctypes over the ioctl):
              v3a.nbody_advance(bodies, dt, steps)
              v3a.closest_hits(rays, spheres, eps) -> [(index, t), ...]
              v3a.any_hits(rays, spheres, eps)    -> [occluded, ...]
              It converts floats <-> Q23.40, fills buffers, writes CTRL,
              sleeps until the IRQ and reads the results.
Key rule    : offload granularity. One PCIe round trip costs tens of
              microseconds, while one CPython pair/intersection costs
              ~0.4-0.7 microseconds. Calling the card per pair or per ray
              would be 10-100x SLOWER than software. Every job is therefore
              a batch: all NSTEPS of nbody in one call; all rays of one
              recursion level ("wavefront") in one call for raytrace.
              (Same lesson as numpy in section 3: fixed per-call overhead
              must be amortized over many elements.)

```

5.5 Acceleration justification (raytrace)

Why a good candidate: Sphere.intersectionTime is the single hottest kernel (179,457 ray-sphere tests per 100x100 frame). Each test is independent of the others, branch-free until the final comparison and uses only +, -, * and one sqrt: ideal for a streaming pipeline. The rest of the frame (shading, recursion, colour mixing) is irregular, data-dependent control flow and stays in Python.

Measured share (hw/model/raytrace_offload_share.py, Python 3.10: replaying all sphere tests of one frame versus rendering the frame): about 41% of the original and about 48% of the optimized render time.

Software changes: render() becomes wavefront (breadth-first): generate all primary rays -> v3a.closest_hits() -> for each hit build the shadow rays (v3a.any_hits) and reflection rays (next closest hits batch) -> repeat up to depth 4 -> shade in Python. The floor (Halfspace) test stays in software (one dot product) and is combined with the sphere result.

Estimate (assumptions: 100 MHz, measured ~1.00 cycle per test in RTL simulation, ~9 batches per frame, ~26k rays x 48 bytes = 1.2 MB of DMA per frame at ~3.9 GB/s, ~20 us driver overhead per batch):

```

hardware per frame = 179,457 tests x 10 ns + DMA + 9 calls ~ 3 ms
optimized CPython: sphere tests ~48% of the frame
Amdahl limit      : 1 / (1 - 0.48) ~ 1.9x
realistic         : ~1.6-1.8x over the optimized software
                   (batch construction in Python costs back part)

```

Compared to nbody the gain is much smaller: nbody offloads almost the whole loop, raytrace only its hottest kernel. Moving Lambert shading and reflection vectors to hardware as well would raise the offloaded fraction, at the cost of more area and a less reusable design.

Accuracy: RTL results are bit-exact to the golden model; hit/miss and closest-sphere decisions matched the double-precision benchmark for every tested primary and shadow ray (t error ~1e-9). Rare decisions exactly at t ~ +-EPS could in principle differ from double precision, so the image is not guaranteed to be bit-identical (unlike the software optimization).

- saturate or raise a status flag, at the cost of extra logic per stage.
- nbody body count is limited to NMAX = 8 (state lives in registers so every pair read is single-cycle). More bodies would move the state into BRAM and add a load stage to the pipeline.
- No synthesis or timing closure was run (not required): the frequency numbers are estimates based on one 64x64 multiply per pipeline stage.

6. Conclusion

raytrace is dominated by Python call and object overhead, not by the ray tracing math. Removing no-op type-check calls, hoisting the loop-invariant shadow ray, inlining the vector math on floats and using `__slots__` cut Python calls by 86% and gives a 2.85x speedup under pyperformance with a bit-identical image. What remains is the per-ray intersection and shading arithmetic (179,457 ray-sphere tests per frame), which is exactly the regular, data-parallel kernel targeted by the hardware accelerator in section 5.